ЛАБОРАТОРНА РОБОТА № 2
Unit-тестування асинхронного коду з Jest

Мета: Метою лабораторної роботи є набуття практичних навичок написання автоматизованих тестів для асинхронного TypeScript-коду з використанням фреймворку Jest.

Хід роботи:

Завдання 1. Типізація функцій через generics

Додати generics до функцій з Лабораторної роботи №1, де тип результату залежить від вхідного параметра.

Список функцій: retryOperation, processInBatches, raceWithTimeout.

Вимоги:

- Змінити сигнатури та внутрішні типові анотації — логіку не змінювати.
- Переконатися, що TypeScript виводить тип результату без явного вказання:
- const result = await retryOperation(op, 3) — тип result має визначитися автоматично.

Лістинг src/functions/retryOperation.ts:

```
import {delay} from './delay.js';
import {AsyncOperation} from '../types/index.js';

export const retryOperation = async <T>(
  operation: AsyncOperation<T>,
  maxRetries: number = 3
): Promise<T> => {
  let lastError: Error = new Error('Unknown error');
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      console.log(`Спроба ${attempt}...`);
      return await operation();


    } catch (error) {
      lastError = error instanceof Error ? error : new Error(String(error));

      if (attempt < maxRetries) {
        await delay(150);
```

```
            }
        }
    }
    throw lastError;
};
```

Лістинг src/functions/raceWithTimeout.ts:

```
import { TimeoutError } from '../types/index.js';
export const raceWithTimeout = async <T>(
    promise: Promise<T>,
    timeoutMs: number
): Promise<T> => {
    const timeoutPromise = new Promise<never>((_, reject) => {
        setTimeout(() => {
            const error = new Error(`Operation timed out after ${timeoutMs}ms`) as TimeoutError;
            error.timeoutMs = timeoutMs;
            reject(error);
        }, timeoutMs);
    });
    return await Promise.race([promise, timeoutPromise]);
};
```

Лістинг src/functions/processInBatches.ts:

```
import { BatchProcessor } from '../types/index.js';

export const processInBatches = async <T>(
    items: T[],
    batchSize: number,
    processor: BatchProcessor<T>
): Promise<T[]> => {
    if (items.length === 0) {
        return [];
    }

    const results: T[] = [];
    const totalBatches = Math.ceil(items.length / batchSize);

    for (let i = 0; i < items.length; i += batchSize) {
        const batchNumber = Math.floor(i / batchSize) + 1;
        const batch = items.slice(i, Math.min(i + batchSize, items.length));

        console.log(`Обробка партії ${batchNumber}/${totalBatches}...`);

        if (batchNumber === 1) {
            results.push(...batch);
        } else {
            const processedBatch = await processor(batch);
            results.push(...processedBatch);
        }
    }
```

```
        return results;
};
```

Зміни в types/index.tx:

```
export type BatchProcessor<T> = (batch: T[]) => Promise<T[]>;
```

Результат виконання:

```
PS D:\Work\Node.js\Lab02> npm start

> lab01@1.0.0 start
> node dist/index.js

1. Тестування delay:
  Початок затримки...
  Затримка 500мс + 15мс компенсації завершена

2. Тестування fetchUserProfiles:
  Отримані профілі:
    ID: 101, Ім'я: User 101, Email: user_101@test.ua
    ID: 102, Ім'я: User 102, Email: user_102@test.ua
    ID: 103, Ім'я: User 103, Email: user_103@test.ua
    ID: 104, Ім'я: User 104, Email: user_104@test.ua
    ID: 105, Ім'я: User 105, Email: user_105@test.ua

3. Тестування retryOperation:
Спроба 1...
Спроба 2...
Спроба 3...
  Результат після 3 спроб: Операція виконана успішно!

4. Тестування processInBatches:
  Вхідний масив: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
  Перша партія буде без змін (контрольна група)
Обробка партії 1/4...
Обробка партії 2/4...
Обробка партії 3/4...
Обробка партії 4/4...
  Результат обробки: [1, 2, 3, 8, 10, 12, 14, 16, 18, 20]

5. Тестування raceWithTimeout:
  Тест 1 (швидка операція, timeout 200ms):
    Успіх: Швидка операція завершена
  Тест 2 (повільна операція, timeout 200ms):
    Таймаут (очікувано): Operation timed out after 200ms
    timeoutMs: 200
```

Рис. 1. Результат виконання завдання 1.

Завдання 2: Налаштування проєкту для тестування

Налаштувати середовище для запуску Jest-тестів з TypeScript для функцій з Лабораторної роботи №1.

Вимоги:

• Встановити залежності: npm install -D jest ts-jest @types/jest.

• Створити jest.config.js з preset ts-jest та testEnvironment: 'node'.

• Додати до package.json скрипти: "test", "test:watch", "test:coverage".

• Переконатися, що команда npm test запускається без помилок.

Мінімальна структура jest.config.js:

```
module.exports = {
preset: 'ts-jest',
testEnvironment: 'node',
};
```

Лістинг jest.config.ts:

```
import type { Config } from 'jest';

const config: Config = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  testMatch: ['**/*.test.ts'],
  extensionsToTreatAsEsm: ['.ts'],
  moduleNameMapper: {
    '^(\\.{1,2}/.*)\\.js$': '$1',
  },
  transform: {
    '^.+\\.tsx?$': ['ts-jest', {
      useESM: true,
    }],
  },
};
export default config;
```

Результат виконання:

```
PS D:\Work\Node.js\Lab02> npm test

> lab02@1.0.0 test
> jest

No tests found, exiting with code 1
Run with `--passWithNoTests` to exit with code 0
In D:\Work\Node.js\Lab02
  18 files checked.
  testMatch: **/*.test.ts - 0 matches
  testPathIgnorePatterns: \\node_modules\\ - 18 matches
  testRegex:  - 0 matches
Pattern:  - 0 matches
```

Рис. 2. Результат виконання завдання 2.

Завдання 3: Тести для fetchUserProfiles

Написати тести для функції fetchUserProfiles з Лабораторної роботи №1.

Сигнатура:

fetchUserProfiles(userIds: string[]): Promise<UserProfile[]>

Вимоги:

- Мінімальна кількість тестів: 3.

Лістинг functions/fetchUserProfiles.test.ts:

```
import { jest, describe, it, expect, beforeEach } from '@jest/globals';
import { fetchUserProfiles } from './fetchUserProfiles.js';

jest.mock('./delay.js', () => ({
  delay: jest.fn<() => Promise<void>>().mockResolvedValue(undefined)
}));

describe('fetchUserProfiles', () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  it('should return array of users', async () => {
    const userIds = ['101', '102', '103'];
    const result = await fetchUserProfiles(userIds);
    expect(result).toHaveLength(3);
  })
  it('should not be empty', async () => {
    const result = await fetchUserProfiles([]);
    expect(result).toEqual([]);
  })
  it('should return correct profile format', async () => {
    const result = await fetchUserProfiles(['101']);
    expect(result[0]).toEqual({
      id: '101',
      name: 'User 101',
      email: 'user_101@test.ua'
    });
  });
})
```

```
PS D:\Work\Node.js\Lab02> npm test

> lab02@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js

(node:21884) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  src/functions/fetchUserProfiles.test.ts
  fetchUserProfiles
    √ should return array of users (175 ms)
    √ should not be empty (1 ms)
    √ should return correct profile format (77 ms)

Test Suites: 1 passed, 1 total
Tests:       3 passed, 3 total
Snapshots:   0 total
Time:        0.58 s, estimated 1 s
Ran all test suites.
```

Рис. 3. Результат виконання завдання 3.

Завдання 4: Тести для retryOperation

Написати тести для функції retryOperation з Лабораторної роботи №1.

Сигнатура:

retryOperation<T>(

operation: () => Promise<T>,

maxRetries: number

): Promise<T>

Вимоги:

- Мінімальна кількість тестів: 3.

Лістинг functions/retryOperation.test.ts:

```
import { jest, describe, it, expect, beforeEach } from '@jest/globals';
import { retryOperation } from './retryOperation.js';

jest.mock('./delay.js', () => ({
  delay: jest.fn<() => Promise<void>>().mockResolvedValue(undefined)
}));

describe('retryOperation', () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  it('should retry and return result after failure', async () => {
    const operation = jest.fn<() => Promise<string>>()
      .mockRejectedValueOnce(new Error('fail'))
      .mockResolvedValue('success');

    const result = await retryOperation(operation, 3);

    expect(result).toBe('success');
    expect(operation).toHaveBeenCalledTimes(2);
  });
  it('should wrap non-Error rejections in an Error', async () => {
    const operation = jest.fn<() => Promise<string>>()
      .mockImplementation(() => Promise.reject('oops'));

    await expect(retryOperation(operation, 1)).rejects.toThrow('oops');
  });
  it('should throw after all retries exhausted', async () => {
    const operation = jest.fn<() => Promise<string>>()
      .mockRejectedValue(new Error('permanent fail'));

    await expect(retryOperation(operation, 3)).rejects.toThrow('permanent fail');
    expect(operation).toHaveBeenCalledTimes(3);
  });

  it('should use default maxRetries of 3', async () => {
```

```
        const operation = jest.fn<() => Promise<string>>()
          .mockRejectedValue(new Error('fail'));

        await expect(retryOperation(operation)).rejects.toThrow('fail');
        expect(operation).toHaveBeenCalledTimes(3);
      });
});
```

Завдання 5: Тести для processInBatches

Написати тести для функції processInBatches з Лабораторної роботи №1.

Сигнатура:

processInBatches<T, R>(

items: T[],

batchSize: number,

processor: (batch: T[]) => Promise<R[]>

): Promise<R[]>

Вимоги:

- Мінімальна кількість тестів: 3.

Лістинг functions/processInBatches.test.ts:

```
import { jest, describe, it, expect, beforeEach } from '@jest/globals';
import { processInBatches } from './processInBatches.js';

describe('processInBatches', () => {
  beforeEach(() => {
    jest.clearAllMocks();
  });

  it('should return empty array when items is empty', async () => {
    const processor = jest.fn<(batch: number[]) => Promise<number[]>>();

    const result = await processInBatches([], 3, processor);

    expect(result).toEqual([]);
    expect(processor).not.toHaveBeenCalled();
  });

  it('should return first batch without processing', async () => {
    const processor = jest.fn<(batch: number[]) => Promise<number[]>>()
      .mockResolvedValue([20, 40, 60]);

    const result = await processInBatches([1, 2, 3, 4, 5, 6], 3, processor);

    expect(result[0]).toBe(1);
    expect(result[1]).toBe(2);
    expect(result[2]).toBe(3);
  });
```

```
    it('should process subsequent batches with processor', async () => {
      const processor = jest.fn<(batch: number[]) => Promise<number[]>>()
        .mockResolvedValue([8, 10]);

      const result = await processInBatches([1, 2, 3, 4, 5], 3, processor);

      expect(processor).toHaveBeenCalledTimes(1);
      expect(result).toHaveLength(5);
    });

    it('should return correct total length', async () => {
      const processor = jest.fn<(batch: string[]) => Promise<string[]>>()
        .mockResolvedValue(['X', 'X', 'X']);

      const result = await processInBatches(
        ['a', 'b', 'c', 'd', 'e', 'f'],
        3,
        processor
      );

      expect(result).toHaveLength(6);
    });
});
```

Завдання 6: Тести для raceWithTimeout

Написати тести для функції raceWithTimeout з Лабораторної роботи №1.

Сигнатура:

raceWithTimeout<T>(

promise: Promise<T>,

timeoutMs: number

): Promise<T>

Вимоги:

- Мінімальна кількість тестів: 2.

Лістинг functions/raceWithTimeout.test.ts:

```
import { jest, describe, it, expect, beforeEach } from '@jest/globals';
import { raceWithTimeout } from './raceWithTimeout.js';

describe('raceWithTimeout', () => {
  beforeEach(() => {
    jest.clearAllMocks();
    jest.useRealTimers();
  });

  it('should return result when promise resolves before timeout', async () => {
    const promise = Promise.resolve('success');

    const result = await raceWithTimeout<string>(promise, 1000);
```

```
    expect(result).toBe('success');
  });

  it('should throw TimeoutError when promise takes too long', async () => {
    const promise = new Promise<string>(resolve =>
      setTimeout(() => resolve('late'), 500)
    );

    await expect(raceWithTimeout<string>(promise, 50))
      .rejects.toThrow('Operation timed out after 50ms');
  });

  it('should include timeoutMs in error', async () => {
    const promise = new Promise<string>(resolve =>
      setTimeout(() => resolve('late'), 500)
    );

    try {
      await raceWithTimeout<string>(promise, 50);
    } catch (error) {
      expect((error as any).timeoutMs).toBe(50);
    }
  });
});
```

Результат виконання тестів:

```
----------------------|---------|----------|---------|---------|-------------------
File                  | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------------|---------|----------|---------|---------|-------------------
All files             |     100 |      100 |     100 |     100 |
 delay.ts             |     100 |      100 |     100 |     100 |
 fetchUserProfiles.ts |     100 |      100 |     100 |     100 |
 processInBatches.ts  |     100 |      100 |     100 |     100 |
 raceWithTimeout.ts   |     100 |      100 |     100 |     100 |
 retryOperation.ts    |     100 |      100 |     100 |     100 |
----------------------|---------|----------|---------|---------|-------------------

Test Suites: 4 passed, 4 total
Tests:       14 passed, 14 total
Snapshots:   0 total
Time:        1.929 s, estimated 2 s
```

Рис. 4. Результат виконання завдань 4-6.

Посилання на репозиторій - **https://github.com/JanRizhenko/node.js-practice**

Висновок: Під час виконання лабораторної роботи були набуті практичні навички написання автоматизованих тестів для асинхронного TypeScript-коду з використанням Jest. Було налаштовано середовище тестування з ts-jest, типізовано функції через generics та написано тести для чотирьох функцій: fetchUserProfiles, retryOperation, processInBatches і raceWithTimeout. Досягнуто 100% покриття коду за statements, functions та lines, що підтверджує якість написаних тестів і коректність реалізації функцій.